



Security Controls Assessment and Evidence Management Platform for GitHub Repositories

Graduation Project 1
Progress Report for the First Seminar

Faculty of Artificial Intelligence Engineering
Department of Software Engineering and Intelligent Information Systems

Prepared by

عمر فوزي العكله

Supervised by

د. كادان جمعة

م. عبدالرحمن عرار

Damascus
Academic Year 2026-2027
October 2026

Progress Summary

This report presents the work completed before the first seminar: defining the problem and scope, reviewing related research and tools, selecting the architecture, and testing a small local model. The diagrams show the proposed solution, and the Gantt chart gives the estimated development schedule.

Completed work	Available output
Problem and scope	Target users, seven core services, 28 principal requirements, and 24 planned checks.
Research and comparison	Five studies and a comparison of assessment, evidence, follow-up, and repair in five tools.
Solution design	Technology choices, architecture, assessment flow, and repair scenario.
Local feasibility	A Python model and stored outcomes for sixteen designed cases.

The integrated application has not been built. The Python model is the only implemented software described here, and its results concern designed local cases. This progress report is prepared for supervisor review.

Contents

Chapter 1 Theoretical Background..... 1
Chapter 2 Project Definition..... 3
Chapter 3 Literature Review and Tool Comparison 5
Chapter 4 Solution Design and Local Results..... 13
References 17

Chapter 1 Theoretical Background

A branch can appear protected while an exempt application can still perform an operation the organization intended to block. Understanding this situation requires separating the security requirement, the evidence used to assess it, and the rules that apply to each actor.

1.1 Security controls and assessment

A security control is a measure intended to protect a system or reduce risk. An automated check answers a specific question about that control. For example, it may confirm that a branch requires a pull request (PR). Whether a particular application can bypass that requirement is a separate question. Neither result establishes compliance with a complete security standard.

NIST SP 800-53A distinguishes examination, interview, and testing as assessment methods [1]. Reading GitHub settings examines technical evidence. Performing an authorized operation under controlled conditions tests behavior. The platform design currently covers configuration assessment; the implemented local model analyzes simplified rule behavior. Validation against live GitHub remains necessary.

Each check needs a defined scope, expected condition, observations, and decision rule. The design distinguishes satisfied, not satisfied, indeterminate, and not applicable. Missing permission or incomplete collection must not be interpreted as a successful check. A conclusive violation can still be recorded when unrelated observations are missing.

1.2 Rules, identities, and exceptions

GitHub rulesets contain requirements for selected branches or tags. Multiple rulesets can apply to one branch, including inherited layers, and can coexist with classic branch protection [2]. A bypass exception belongs to its own ruleset. Bypassing one layer does not automatically bypass another.

An actor is a person or automation identity. A bot in this project is an application operating with authorized repository access. Team and role memberships affect which exceptions apply. The model must therefore keep rules, selectors, and exceptions associated with their original layers.

A normal fast-forward update advances a branch along its existing history. A non-fast-forward update can replace that history. An organization may permit a release application to perform normal updates while forbidding history rewriting. Pull-request review and required-check conditions introduce separate restrictions [3]. These distinctions are the basis of the repair example in Chapter 4.

The API also imposes evidence limits. GitHub documents that ruleset bypass details are returned only when the requesting identity has write access to that ruleset [4]. An omitted bypass field cannot be treated as an empty exception list.

1.3 Constraint-based repair

The organization specifies actor-operation pairs that must remain allowed and others that must be denied. It also specifies which layers may change. The engine searches a finite set of permitted edits and rejects candidates that violate these conditions.

Let E(C,a,o) be the modeled decision for configuration C, actor a, and operation o. It returns ALLOW, DENY, or UNKNOWN. A repair C′ must meet both sets of requirements:

$$\forall (a, o) \in P_{allow} : E(C', a, o) = ALLOW$$

$$\forall (a, o) \in P_{deny} : E(C', a, o) = DENY$$

The local search first prefers fewer ruleset writes, then fewer new rulesets, then fewer changes to modeled fields. Creating or updating one ruleset counts as one write. The search also checks whether the edits can be ordered without violating requirements that already hold. A two-write search can miss a valid three-write repair. Repeatable output alone does not establish that the model matches GitHub.

1.4 Evidence and assessment history

Evidence provenance records the source, time, repository, branch, access context, and collection limitations. A digest detects changes to stored data; it does not prove that the data was complete or truthful. Multiple API calls also do not form an atomic snapshot.

Re-verification must create a new assessment while retaining the original evidence and policy version. Closing a remediation task is an administrative action, separate from proving that a technical condition has changed. The proposed data flow preserves that distinction.

Chapter 2 Project Definition

2.1 Problem and objective

Repository administrators need to assess security requirements, understand failures, retain evidence, and verify the result after a repair. Existing tools already perform many individual checks. This project adds a narrower task to that workflow: finding a repair for layered GitHub rules while keeping the operations the organization explicitly needs.

The objective is to build a Security Controls Assessment and Evidence Management Platform for GitHub Repositories. The intended users are repository administrators, security evaluators, and reviewers in software-development organizations. The work completed so far defines its scope, requirements, architecture, and proposed contribution.

2.2 Agreed scope

The selected scope is GitHub.com repositories accessible through an authorized connection. Branch assessment starts with an existing default branch. Repository-wide settings retain their own scope. The platform collects configuration and security-service metadata rather than cloning repository source code.

The target catalog contains 20 core checks and four conditional checks. It covers branch protection, approved exceptions, Actions permissions and sources, writable deploy keys, repository visibility, webhook transport, and evidence from available GitHub security services [3]-[6], [20]-[24]. The catalog defines planned coverage; implementation has not begun.

Table 2.1 Services defined in the scope

Service	Intended outcome
Scope and policy	Select repositories, questions, expected conditions, and policy version.
Connection and collection	Obtain authorized observations and expose collection limits.
Assessment	Save a run with explicit decisions and explanations.
Policy-effect analysis	Explain supported actor-operation decisions across layers.
Repair planning	Produce reviewable changes and predicted effects.
Remediation and history	Assign follow-up work and re-verify it without replacing old results.
Evidence and reporting	Retrieve supporting records and produce an assessment report.

The requirements inventory contains 28 principal functional requirements: 22 across these services and six supporting identity, authorization, audit, progress, and recovery functions. One principal requirement contains the 24-check catalog. The counts describe different levels of decomposition; they do not measure equal implementation effort.

Automatic changes to GitHub, other hosting providers, broader governance and compliance services, and new source-code or dependency vulnerability scanners are outside the initial scope. Existing alerts will be consumed as evidence. Retrieval-augmented generation (RAG) is deferred and is not required by the repair engine.

2.3 Contribution selected for evaluation

The proposed contribution is a repair engine for a bounded set of layered GitHub policies. The organization specifies which operations each actor needs, which operations must be blocked, and which rulesets may change. The engine generates possible edits, checks their effects, and selects a plan using an explicit cost. Instead of stopping at a failed check, it tries to produce a repair that meets those needs.

The research question is whether this method can generate correct plans within its model while preserving required operations, and how the plans compare with correct manual or scripted repairs. Earlier research already addresses policy repair. Our contribution is the GitHub-specific method and its evaluation; an advantage over correct manual work remains to be demonstrated.

The initial model covers active layers, known memberships, always bypass, a simplified approved-PR operation, and non-fast-forward restrictions. Relevant unsupported rules or classic protection cannot be silently omitted. Broader interpretation and live validation remain open work.

Chapter 3 Literature Review and Tool Comparison

3.1 Review scope

Five studies support different parts of the project. CloudFix and IAM-PolicyRefiner inform the repair method. Automated Security Findings Management, the study named in the project application, informs findings management. Towards Automated Continuous Security Compliance explains evidence and automation challenges. A GitHub Actions study motivates permission checks. Their findings are evidence from their own settings, not measurements of our platform.

3.2 CloudFix 2026

CloudFix is a January 2026 preprint on repairing AWS IAM access-control policies [7]. Its input is an existing policy and requests labelled must-allow or must-deny. A request includes an identity, action, resource, and environment conditions. The task is to correct wrong access decisions while satisfying both groups of requests. The paper's own motivating example adds permission for an operation that an existing policy unintentionally denies.

The method has four components. The goal validator encodes policies and requests using Quacky and SMT reasoning. If a request is misclassified, the fault localizer identifies relevant policy statements. A prompt generator passes this information to an LLM, which proposes a replacement policy. The validator checks the candidate again. The experiment permits five iterations; when no complete repair is found, the method retains the candidate with the highest request-classification accuracy. Such an output is a partial repair, not an accepted solution to every requirement.

The evaluation uses 282 policies collected from forum posts, augmented with constructed request sets of 10, 20, 30, or 50 requests per policy. The baseline receives the policy, required decisions, and repair instructions without the fault-localization report. At ten requests, the guided method completely repairs 237 policies, compared with 136 for the baseline. Its mean request accuracy is 95.21%; that figure differs from its 84.0% complete-policy repair rate. At fifty requests, only seven guided repairs are complete, compared with zero for the baseline. The mean-accuracy difference at that size does not reach the paper's 0.05 significance threshold. These results expose a limitation as specifications grow, rather than establishing dependable repair for arbitrary policies. The figures are reported in Tables II and III.

For our project, the useful lesson is to check every declared allow and deny objective and distinguish complete satisfaction from a partially improved candidate. We adopt that evaluation discipline, while using a deterministic search over supported GitHub edits. We do not inherit CloudFix's results or claim that deterministic generation itself is novel. The required GitHub semantics and the practical value of our plans still need independent validation.

3.3 IAM-PolicyRefiner 2024

Automatically Reducing Privilege for Access Control Policies, published in OOPSLA 2024, presents IAM-PolicyRefiner [8]. It takes an AWS policy and observed access requests.

Requests are associated with matching policy statements, and their values are generalized within a defined space of local edits. The aim is a least-permissive refinement that preserves the recorded access and remains inspectable. Its teaching-assistant example narrows permitted actions and file paths while retaining needed uploads. This is a semantic restriction objective, distinct from minimizing our number of ruleset writes.

The evaluation uses eleven roles from four AWS accounts and three months of CloudTrail logs, with 70,000-320,000 requests per role. The reported processing time is under one minute per role. With half the log used for refinement, all requests pass for every evaluated role. Smaller samples can omit necessary behavior: for one role, refinement from the first 10% allows only 62% of all requests. Access Analyzer checks support that the computed policies are no more permissive than the originals.

Section 6 also introduces IAM-PolicyRefinerNeg, which adds a deny statement derived from forbidden requests. In Section 7.2, a coarse prefix abstraction initially denies everything on the evaluated cases; a more precise string abstraction avoids this outcome. Thus both preservation and the choice of representation matter. Two roles also violate an assumption needed for the unique-minimum result, so that theoretical claim requires care.

Our design implication is to specify the supported edit space and preservation requirements explicitly, and to test whether its representation causes excessive denial. We take required operations directly from the organization. We claim neither invention of repair from forbidden requests nor transfer of the paper's guarantees to layered GitHub rules.

3.4 Automated Security Findings Management 2024

Automated Security Findings Management: A Case Study in Industrial DevOps, published in ICSE-SEIP 2024, is the study named in the project application [9]. It presents a methodology instantiated as Security Flama. The problem is broader than finding vulnerabilities: tool reports use different formats and terminology, overlap, and leave teams with repeated manual interpretation and follow-up.

Its processing path collects reports, maps them to a common format, groups semantically similar findings, and enriches them with project-specific rules. It records first and last occurrences, source distribution, severity, priority, and status. An API and web views communicate results according to stakeholder roles. The method distinguishes states including Open, In Work, False Positive, Accepted, Solved, and Disappeared. A finding absent from a later report can become Disappeared automatically; that state is separate from a user marking it Solved. These details are described in Sections 3.1-3.5.

The industrial evaluation covers two DevOps projects and combines sixteen quantitative collection occasions with repeated interviews. In the studied projects, aggregation reduces the displayed number of findings by at least two-thirds. The substantial reduction of unresolved findings occurs in one project; the other shows marginal improvement, linked to limited adoption. Participants value an accessible, current overview, but report duplicate maintenance alongside the backlog, difficult manual prioritization, and insufficient visibility into processing. These findings appear in Sections 4 and 5. They do not establish a general time-saving percentage or an equally strong effect in every team.

For our platform, this supports preserving source and occurrence history, explaining status changes, and separating administrative closure from technical re-verification. It also gives us evaluation questions: can users understand why a finding changed, and does follow-up duplicate their existing work? Our first version does not reproduce Security Flama's semantic aggregation or knowledge-base architecture. Ordinary findings management is established prior work; our repair-engine evaluation remains a separate contribution.

3.5 Revisiting Security Practices 2025

Revisiting Security Practices for GitHub Actions Workflows, published at ICPC 2025, asks how prevalent workflow security issues are and how they changed between 2022 and 2024 [10]. The study compares the same 18,938 workflows from 5,246 repositories, excluding newly added and unavailable workflows to keep the temporal comparison meaningful. It adapts GHAST to detect confidential-data disclosure, command injection, unverified actions without a full commit pin, and missing explicit token permissions. Table I defines these categories.

The detector reports 33,362 instances in the 2024 versions: 2,731 vulnerability instances and 30,631 misconfigurations. These are tool findings, not counts of successful attacks or independent affected repositories. The share of workflows without explicit permissions falls from 84.7% to 74.9%. However, the reported confidential-data and command-injection instances increase; their changes are not statistically significant in the paper. The results therefore do not support a blanket statement that workflow security improved. Table III and Figure 1 separate the categories and time periods.

The selection favors active, popular public repositories, and results depend on the detector and its definitions. They cannot be generalized to all GitHub repositories. Missing a permissions declaration also does not, by itself, prove that a token received excessive effective access.

For our initial catalog, the study supports explicit questions about Actions permissions and sources. Our repository-configuration checks do not inspect every workflow or reproduce these four analyses. A safe repository default cannot certify every workflow, and our report must state that limit. Workflow-file analysis would require a separately scoped extension.

3.6 Towards Automated Continuous Security Compliance 2024

Towards Automated Continuous Security Compliance is an ESEM 2024 vision paper, rather than an evaluation of a completed compliance product [11]. It develops a definition, organizes challenges, and proposes a research roadmap. It treats compliance requirements as coming from sources such as standards, laws, internal policies, and contracts, with assessment integrated into the development lifecycle. It also warns against choosing an automation solution before understanding its requirements.

The challenge study searches IEEE Xplore, ACM Digital Library, and Scopus. From 110 initial hits, filtering and snowballing retain four manuscripts containing 62 challenges. The analysis corroborates nine of fifteen previously identified industrial challenges and groups additional material into twelve new challenges. A preliminary interview with one experienced

security expert checks understandability and priorities; it is not broad industrial validation. Sections 2.1 and 2.2 explain these methods and limits.

Three challenges directly inform our design. NC2 concerns changes in requirements, design, or implementation breaking security requirements. NC4 concerns the effort of generating and documenting evidence. NC7 concerns limitations of security tests. The roadmap therefore helps justify traceable assessment questions, evidence collection, and renewed assessment after relevant changes. It does not supply measured savings or prove that automated tests establish complete compliance.

Our concrete response is to retain the scope, policy version, collection time, and limitations behind each judgment, then create a new assessment after a repair. For example, an old branch-settings record cannot establish the state after a new exception is added. That example is our application of the paper's challenges, not an experiment reported by its authors. We use the paper to justify requirements and evaluation boundaries, without treating its roadmap as a competing repair engine.

3.7 Research synthesis

Table 3.1 Research methods and concrete design implications

Study	Problem and method	Project decision and boundary
CloudFix 2026 [7]	Repair wrong AWS decisions using labelled requests, guided LLM proposals, and SMT checking.	Check every declared allow and deny objective before accepting a repair. Guarantees remain relative to the model and requests.
IAM-PolicyRefiner 2024 [8]	Tighten a policy within readable modifications while retaining logged access.	Define permissible edits and preserve required operations. Our edit cost is not the paper's least-privilege objective.
Security Flama 2024 [9]	Normalize, aggregate, track, and communicate findings; evaluate in two industrial projects.	Preserve source, history, and meaningful status. Test clarity and user effort; these functions are established prior work.
Revisiting Security Practices 2025 [10]	Compare workflow practices over time with a detector.	Include permission questions and state the limits of repository-level settings. Workflow code analysis remains outside initial scope.
Continuous Security Compliance 2024 [11]	Analyze compliance challenges and propose an automation roadmap.	Define evidence requirements and re-assessment conditions. This is problem framing, not a product-performance experiment.

The first two studies are direct methodological prior work. Security Flama is direct application prior work for the platform workflow. The remaining studies support problem selection and assessment boundaries. None establishes that our full platform outperforms

existing tools. The next comparison therefore uses correct manual or scripted plans and the same explicit objectives, rather than treating shared workflow features as novelty.

3.8 Tool selection and comparison method

Five tools were selected for their relationship to the platform's tasks. Scorecard, Checkov, Legitify, and Minder provide direct comparisons for GitHub security assessment and remediation. DefectDojo Open Source provides a comparison for handling findings, supporting files, and remediation history. It is included because the project also manages evidence and follow-up, rather than to treat a findings manager as a repository scanner.

The comparison covers scope, policy customization, result detail, supporting evidence, findings and reports, rule layering and exceptions, repair objectives, plan generation and checking, and execution or re-verification. Capability overviews use official documentation reviewed on 28 October 2026 [25]-[31]. The narrower implementation claims retain the four pinned source paths [12]-[15]. DefectDojo is a documentation review of the Open Source workflow; its Pro edition and extensions are outside this comparison. The reviewed products have not been exercised here as complete running systems.

Descriptions state what the source supports. A capability not verified is recorded as not verified, rather than absent. Customization, external scripts, and integrations can extend the reviewed tools. The project entries distinguish a planned platform from its implemented local model.

3.9 Platform coverage and operational workflow

Table 3.2 Tool coverage policies outputs and follow-up

Tool	Relevant scope	Policy and result model	Evidence and follow-up
Scorecard [12], [25]	Repository practices, branch protection, and Actions checks.	Check criteria and configurable score thresholds; scores, reasons, and JSON output.	Detailed findings and remediation guidance can feed another workflow; output alone does not establish our proposed evidence provenance.
Checkov [13], [26]	GitHub configuration and separate workflow-file scanning.	Built-in and custom policies; Python or YAML support depends on the runner and policy type.	Check reports and output formats including SARIF. The reviewed GitHub runner is not evidence of an integrated case-management lifecycle.
Legitify [14], [27]	Organization, repository, member, Actions, and runner-group policies.	OPA policies, selectable scope, severity-based results, JSON, and SARIF.	Findings and policy guidance support follow-up; this comparison concerns Legitify Open Source, not commercial Legit Security.

Tool	Relevant scope	Policy and result model	Evidence and follow-up
Minder [15], [28]	Repository and pull-request policies, with multiple ingestion sources.	Profiles, parameters, and evaluators including Rego and JQ.	Policy results, alerting, and remediation; the framework supports PR and REST actions.
DefectDojo OSS [29]-[31]	Findings imported from security tools and organized by assets and tests.	Finding states, risk acceptance, and configured triage workflows.	Supporting files, notes, mitigation state, dashboards, and reports. These are existing capabilities, not project novelty.
Our platform, planned	Twenty core and four conditional GitHub checks, with a bounded repair component.	Institutional policy, four assessment outcomes, and explicit actor-operation objectives.	Source-linked evidence, history, remediation, re-verification, reports, and access control are designed but unimplemented.

DefectDojo can attach supporting files to Findings, Engagements, and Tests, with permission checks on retrieval [30]. File attachments and remediation states are therefore shared baseline capabilities. Evidence provenance in our design means recording collection source, time, scope, policy and evaluator version, and known collection gaps. That proposed structure still needs implementation and evaluation; its definition is not evidence that every competitor lacks it.

3.10 Repair behavior and the proposed difference

Table 3.3 Reviewed repair inputs outputs and execution

Tool or path	Input and rule interpretation	Repair output and checking	Execution and assessment boundary
Scorecard Branch-Protection [12]	Existing branch protection and ruleset information; the policy interface sets check thresholds.	Findings, scores, and guidance in this path; it does not take our actor-operation objective list to synthesize an edit plan.	Guidance for the maintainer; this statement does not cover other Scorecard patch features.
Checkov GitHub Config. [13]	Classic branch-protection data in the pinned collection path, evaluated by checks.	Reports on the collected configuration; this path does not synthesize Ruleset edits from actor objectives.	Remediation guidance; separate runners and commercial integrations are not assessed here.
Legitify repository policies [14]	Includes Rulesets and bypass checks, with customizable OPA policies.	The reviewed bypass guidance recommends clearing the list. It does not check that advice against our declared required operations.	The maintainer follows the guidance. Custom Rego or an external planner can add behavior.

Tool or path	Input and rule interpretation	Repair output and checking	Execution and assessment boundary
Minder OSPS-AC-03.01 [15], [28]	A ready-made non-fast-forward check and a configurable framework.	The reviewed metadata supplies a uniform blocking template. Framework evaluation output can feed remediation; the template itself does not search our costed objective space.	PR or REST remediation is available. Minder already offers execution that our first version leaves manual.
DefectDojo OSS [29]	Imported findings, mitigation guidance, and workflow decisions.	This review did not verify a native interface that synthesizes GitHub Ruleset repairs from an actor-operation contract.	Tracks remediation status. Closing a finding is not itself a live GitHub verification of the required operations.
Our local model	Explicit allow/deny objectives, known actor memberships, active supported layers, and editable or fixed boundaries.	Bounded candidate search, checking of every declared objective, and cost-based selection with before/after effects.	A local Python model exists. Application in GitHub is manual in the first-version design; live verification remains pending.

The specific target is to generate a repair that satisfies the declared actor-operation objectives while respecting fixed layers and a stated modification cost. In the release-and-synchronization example in Chapter 4, a blanket force-push block meets one objective but stops required synchronization. A valid plan preserves that operation while blocking the release application's force push. The comparison is about this defined task, not about counting unrelated product features.

A knowledgeable administrator or a tailored script can produce the same valid plan. They are therefore strong evaluation baselines. The stored local results show equal write counts for the eight feasible or already-satisfied cases; they do not demonstrate less user effort or superior whole-product accuracy [18]. A fair next evaluation uses the same input snapshots and objectives, includes incomplete and unsupported cases, and records fulfilled objectives, required operations broken, edit cost, search limits, and independently measured user effort where feasible.

Chapter 4 Solution Design and Local Results

4.1 Selected architecture

The selected architecture is a modular monolith in C#. Web and Worker share modules for identity and authorization, scope and policy, assessment coordination, evidence and history, findings and remediation, and reporting and audit. The rule engine takes validated input and returns decisions or repair plans without calling GitHub or the database. Collection and storage belong to the integration and data-access modules. Arrows point from the caller to the called component. Numbers trace one saved assessment job; responses are omitted.

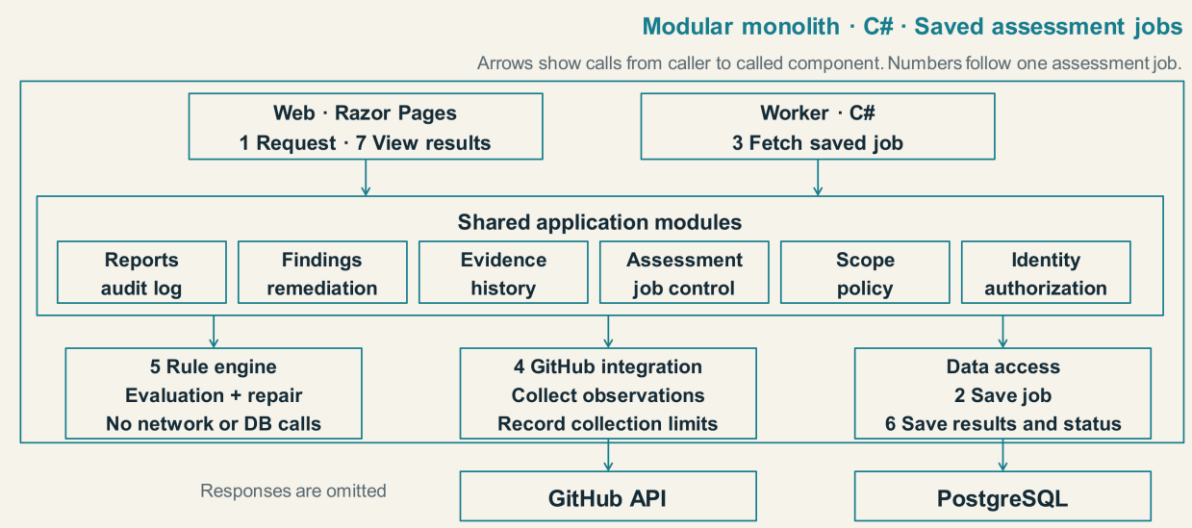


Figure 4.1 Application modules and the saved assessment job

The selected stack is ASP.NET Core 10 with long-term support (LTS), Razor Pages, Bootstrap, PostgreSQL, Entity Framework Core (EF Core), and Npgsql. Published lifecycle and provider documentation support this release combination [16], [17]. The choice fits a single-developer project with related assessment, evidence, and remediation records. Separate network services would add deployment and failure-handling work without a demonstrated present requirement.

Assessments are designed as saved background jobs. A request is accepted after the job and its scope and policy references are saved. The worker obtains saved work through application logic and data access, collects observations, invokes supported checks, and stores evidence, results and job status. Web reads these records to show progress and results. Recovery must preserve attempts and avoid duplicate final records. The durable-job implementation and hosting provider remain undecided. Python remains a local research reference rather than a production backend.

4.2 Assessment workflow

In the designed workflow, a user selects a scope and policy, requests an assessment, and reviews the findings with their evidence. For a supported case, the engine can suggest a repair. The administrator reviews and applies the change in GitHub, then requests re-verification. The diagram describes this design rather than a running application.

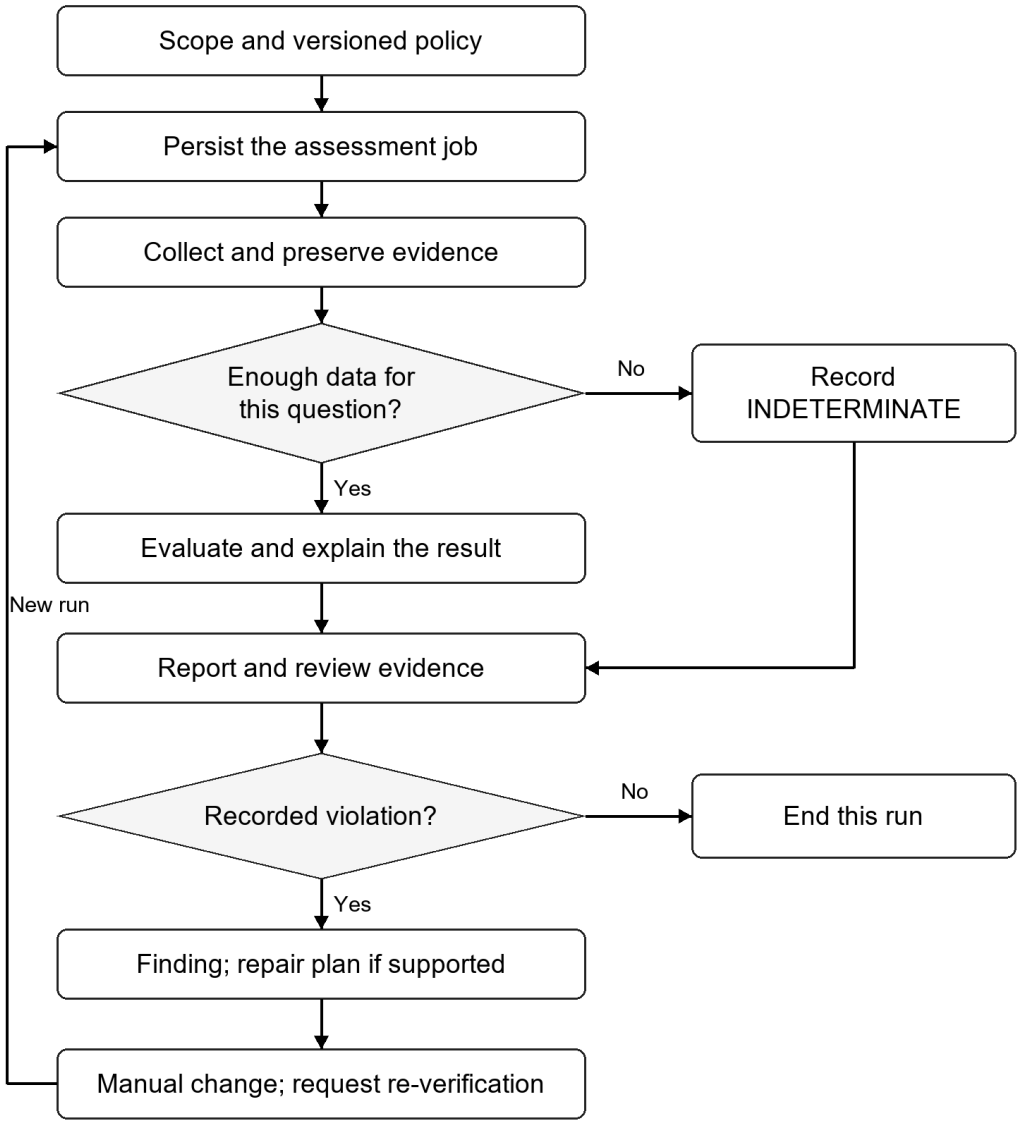


Figure 4.2 Designed assessment and re-verification flow

The design keeps evidence scope, collection errors, and evaluator versions visible. Organization-level authorization applies to stored evidence and reports. A retry must not duplicate final records, and a failed collection request must not become an empty successful result.

4.3 A repair example

Consider R1, a ruleset that requires a pull request and blocks non-fast-forward updates. Both a release application and a synchronization application have an always bypass exception. The organization needs the release application to keep making normal updates but stop rewriting history. Synchronization must keep its approved history-rewriting operation, while a developer must continue to use PR review.

Removing both exceptions would block necessary operations. An unconditional non-fast-forward blocker would stop synchronization. In the model, an additional R2 blocks non-fast-forward updates while exempting synchronization only. R1 stays in place. The release

application can still update normally but cannot rewrite history. This is a constructed example, not a production incident or a live GitHub test.

The engine searches the allowed edits and checks each candidate against the stated requirements. It does not store this example as a fixed answer. A knowledgeable administrator can write the same plan; the component is intended to automate finding, checking, and explaining it for supported cases.

4.4 Preliminary local results

The local Python prototype implements a small ruleset model and a bounded repair search. Its test cases, comparison script, and stored results date from 26 October 2026 [18]. The stored outputs were checked against their corresponding input files while preparing this report.

Table 4.1 Recorded outcomes for sixteen designed local cases

Outcome	Cases
Repaired within the model	7
Already satisfied	1
No plan within the two-write bound	4
Contradictory requirements	1
Incomplete input	1
Unsupported input	2

For the eight cases that could be repaired or already met the requirements, both the engine and the manual reference satisfied the required decisions and used the same number of writes. One case with no plan within two writes has a known three-write repair, illustrating the effect of the search limit.

A separate implementation checked the plans, but it shares the same interpretation of GitHub rules. The cases were designed for the model rather than sampled from organizations. The results show local feasibility. They do not yet establish agreement with live GitHub or a reduction in user effort.

4.5 Status and immediate next work

Completed work comprises problem and scope definition, the 28-requirement inventory and 24-check catalog, five-paper review, five tool comparisons, architecture and workflow design, and the local feasibility artifact. The ASP.NET Core application, database, collector, and live laboratory validation remain unimplemented.

The next step is to validate supported rule semantics in an authorized GitHub laboratory and build one complete assessment path with evidence and re-verification. Later evaluation will compare correct-plan rate, declared-operation regressions, edit cost, search limits, and, if measured, user effort. Development is planned as iterative increments managed with Kanban, consistent with Agile's emphasis on working software and simplicity [19].

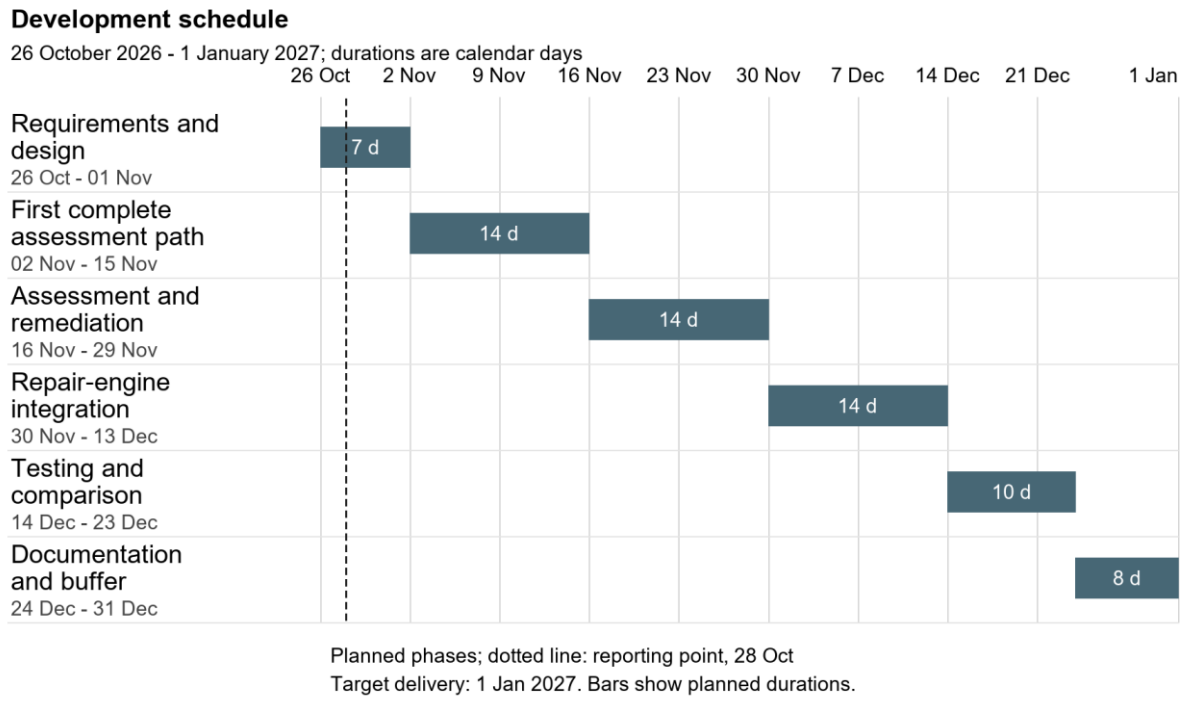


Figure 4.3 Estimated development schedule

The schedule keeps the six phases and dates prepared for the seminar, with target delivery on 1 January 2027. Scope definition, the literature review, architecture, and the local model are complete at the level described here. Live validation in the first phase is still pending. The bars show planned durations; testing also accompanies each implementation increment.

References

[1] Joint Task Force, “Assessing Security and Privacy Controls in Information Systems and Organizations,” NIST SP 800-53A Rev. 5, 2022.

https://doi.org/10.6028/NIST.SP.800-53Ar5

[2] GitHub, “About rulesets,” GitHub Docs. Accessed Oct. 27, 2026.

GitHub Docs: online documentation

[3] GitHub, “Available rules for rulesets,” GitHub Docs. Accessed Oct. 27, 2026.

GitHub Docs: online documentation

[4] GitHub, “REST API endpoints for rules,” GitHub Docs. Accessed Oct. 27, 2026.

https://docs.github.com/en/rest/repos/rules

[5] GitHub, “REST API endpoints for GitHub Actions permissions.” Accessed Oct. 27, 2026.

https://docs.github.com/en/rest/actions/permissions

[6] GitHub, “Managing GitHub Actions settings for a repository.” Accessed Oct. 27, 2026.

GitHub Docs: online documentation

[7] B. Hall, O. Ungaro, and W. Eiers, “CloudFix: Automated Policy Repair for Cloud Access Control Policies Using Large Language Models,” arXiv:2512.09957, ver. 2, Jan. 2026, preprint.

https://arxiv.org/html/2512.09957v2

[8] L. D’Antoni, S. Ding, A. Goel, M. Ramesh, N. Rungta, and C. Sung, “Automatically Reducing Privilege for Access Control Policies,” Proc. ACM Program. Lang., vol. 8, OOPSLA2, art. 298, 2024, doi: 10.1145/3689738.

https://www.amazon.science/publications/automatically-reducing-privilege-for-access-control-policies

[9] M. Voggenreiter, F. Angermeir, F. Moyon, U. Schopp, and P. Bonvin, “Automated Security Findings Management: A Case Study in Industrial DevOps,” ICSE-SEIP 2024, pp. 312-322, doi: 10.1145/3639477.3639744.

https://arxiv.org/html/2401.06602v1

[10] J. Huang and B. Lin, “Revisiting Security Practices for GitHub Actions Workflows,” ICPC 2025, doi: 10.1109/ICPC66645.2025.00016.

https://jiangnanpro.github.io/assets/pdf/Huang2025a.pdf

[11] F. Angermeir, J. Fischbach, F. Moyon, and D. Mendez, “Towards Automated Continuous Security Compliance,” ESEM 2024, vision paper. Author manuscript: arXiv:2407.21494.

https://arxiv.org/html/2407.21494v1

[12] OpenSSF, “Scorecard Branch-Protection source and policy interface,” commit 47770053fe5d4445ffedd98754258a24b75792ac. Reviewed Oct. 26, 2026.

https://github.com/ossf/scorecard/blob/47770053fe5d4445ffedd98754258a24b75792ac/checks/branch_protection.go

[13] Checkov, “GitHub configuration runner,” commit 811d0687902c66d5964c68da5739c947979e85b1. Reviewed Oct. 26, 2026.
https://github.com/bridgecrewio/checkov/blob/811d0687902c66d5964c68da5739c947979e85b1/checkov/github/runner.py

[14] Legitify, “GitHub repository policies,” commit 038aa49473a6974a3ef79f6c76b949b689d23282. Reviewed Oct. 26, 2026.
GitHub source (pinned commit)

[15] Minder, “OSPS-AC-03.01 rule and remediation metadata,” commit 9d1841adbf0da3026550671c74b2fcd51558bc95. Reviewed Oct. 26, 2026.
GitHub source (pinned commit)

[16] Microsoft, “.NET and .NET Core support policy.” Accessed Oct. 27, 2026.
https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core

[17] Npgsql, “Npgsql Entity Framework Core Provider 10.0 Release Notes.” Accessed Oct. 27, 2026.
https://www.npgsql.org/efcore/release-notes/10.0.html

[18] Project research records, “Source-bounded comparison protocol,” PROTOCOL.md, cases.json, comparison-results.json, and repair_spike.py, local artifact set, Oct. 26, 2026.

[19] Manifesto for Agile Software Development, “Principles behind the Agile Manifesto,” 2001. Accessed Oct. 27, 2026.
https://agilemanifesto.org/principles.html

[20] GitHub, “REST API endpoints for deploy keys.” Accessed Oct. 27, 2026.
https://docs.github.com/en/rest/deploy-keys/deploy-keys

[21] GitHub, “REST API endpoints for repository webhooks.” Accessed Oct. 27, 2026.
https://docs.github.com/en/rest/repos/webhooks

[22] GitHub, “REST API endpoints for Dependabot alerts.” Accessed Oct. 27, 2026.
https://docs.github.com/en/rest/dependabot/alerts

[23] GitHub, “REST API endpoints for code scanning.” Accessed Oct. 27, 2026.
https://docs.github.com/en/rest/code-scanning/code-scanning

[24] GitHub, “REST API endpoints for repositories.” Accessed Oct. 27, 2026.
https://docs.github.com/en/rest/repos/repos

[25] OpenSSF, “Scorecard project overview, checks, and outputs.” Reviewed Oct. 28, 2026.
https://github.com/ossf/scorecard

[26] Checkov, “Custom Policies Overview.” Reviewed Oct. 28, 2026.
https://www.checkov.io/3.Custom%20Policies/Custom%20Policies%20Overview.html

[27] Legit Labs, “Legitify overview, namespaces, and output formats.” Reviewed Oct. 28, 2026.
https://github.com/Legit-Labs/legitify

[28] Minder, “Rule Evaluation in Depth.” Reviewed Oct. 28, 2026.
https://docs.mindersec.dev/ref/rule_evaluation_details

[29] DefectDojo, “Introduction to Findings,” Open Source workflow. Reviewed Oct. 28, 2026. https://docs.defectdojo.com/triage_findings/findings_workflows/intro_to_findings/

[30] DefectDojo, “Attaching Files (Open Source).” Reviewed Oct. 28, 2026. https://docs.defectdojo.com/triage_findings/findings_workflows/os__add_files/

[31] DefectDojo, “Main Dashboard (Open Source).” Reviewed Oct. 28, 2026. https://docs.defectdojo.com/metrics_reports/dashboards/introduction_dashboard/